Database Schema - Design Decisions - Document

1. Dual-Purpose Keys for One-to-One Relationships

For tables like MochiEntity and IdealDay, the user_id column is configured as **both the Primary Key and the Foreign Key**.

- **The Reason:** This structural choice automatically enforces a strict 1:1 relationship at the database level. A user can never accidentally have two virtual pets or two different daily schedules because duplicate user IDs are prohibited in the primary key column.

2. Safeguarding Shared Assets via Restrictive Deletes

In the Activity table, your creator_id uses a cascading delete, but the Timeslot reference is intentionally protected.

- **The Reason:** While deleting a user should safely wipe out their custom activities, deleting a global time window must not be allowed to break the calendar system. The system forces a halt (RESTRICT) if a database operation tries to delete a time block that existing activities are still anchored to.

3. Composite Identity for Many-to-Many Relationships

The Friendship, ChallengeParticipation, and DailyGoals tables do not use standard auto-incrementing id columns. Instead, they use **Composite Primary Keys** (combining multiple foreign keys together, like PRIMARY KEY (friend1_id, friend2_id)).

- **The Reason:** This completely blocks data duplication. A user cannot join the same challenge twice, send duplicate friend requests, or assign the exact same subgoal to the same day multiple times.

4. Self-Contained Value Constraints (Domain Integrity)

The database uses precise inline CHECK constraints to enforce business rules directly inside the tables rather than relying on application code.

- **Examples:** max_working_hours is locked between 0 and 24, and the Timeslot table includes a check_dates constraint ensuring an end_time can never occur before a start_time. This guarantees the database remains accurate and logical, even if a bug occurs in the backend code.